AROR UNIVERSITY

LEARN • INNOVATE • LEAD

"Building Intelligent Tomorrows"

ARI-406 | BS ARTIFICIAL INTELLIGENCE COURSE

LECTURE 1

Introduction to Generative Artificial Intelligence

IDEAS TODAY • INTELLIGENT TOMORROW



- **Text** — Write & generate content
- **Images** — Create visuals
- **Code** — Build solutions
- **Audio** — Compose sounds
- **Video** — Bring ideas to life

Create • Innovate • Transform

COURSE INSTRUCTOR

Dr. Rajesh Kumar

Assistant Professor
University of New York Tirana

Adjunct Faculty Member
AROR University

Generative AI

- Understanding the Technology Behind ChatGPT, Gemini, Claude & Modern AI
- **Lecture 1: Introduction to Generative AI**
- What is Generative AI?
- How does it work?
- What can it create?
- How is it different from traditional AI?
- Where is it being used?

Lecture Objectives

- **By the end of this lecture, you will understand:**
- What Artificial Intelligence (AI) means
- What Machine Learning (ML) means
- What Generative AI means
- How Generative AI differs from traditional AI
- What Large Language Models (LLMs) are
- How tools such as ChatGPT generate responses
- Real-world applications of Generative AI
- Opportunities and limitations of Generative AI

- **What can AI create?**

- **How can machines create something that did not exist before?**
- **Can a computer write a story, create an image, or compose music?**

Examples:

- ✍️ Text
- 🖼️ Images
- 🎵 Music
- 🎥 Video
- 💻 Computer code
- 🗣️ Speech
- 📊 Presentations
- 🧩 3D models
- **Key message:**
- **Generative AI is AI that can generate new content.**

- **Question:**
- If AI creates a new image, where did that image come from? Did AI invent it, or did it learn from existing examples?

- Example:
- **Human writer:**
- Reads many books
- Learns language and storytelling
- Creates a new story
- **Generative AI:**
- Trains on millions of text examples
- Learns language patterns
- Generates a new story based on a prompt

- Generative AI does not create from nothing. It learns the underlying patterns of existing data and generates new combinations or outputs that were not explicitly present in the training data

What is Artificial Intelligence?

- AI is the field of computer science focused on creating systems that can perform tasks that normally require human intelligence.
- Examples:
- Understanding language
- Recognizing images
- Making predictions
- Solving problems
- Making decisions
- Recognizing speech

AI → Machine Learning → Deep Learning → Generative AI

- The AI Family
- Artificial Intelligence
- ↓
- Machine Learning
- ↓
- Deep Learning
- ↓
- Generative AI

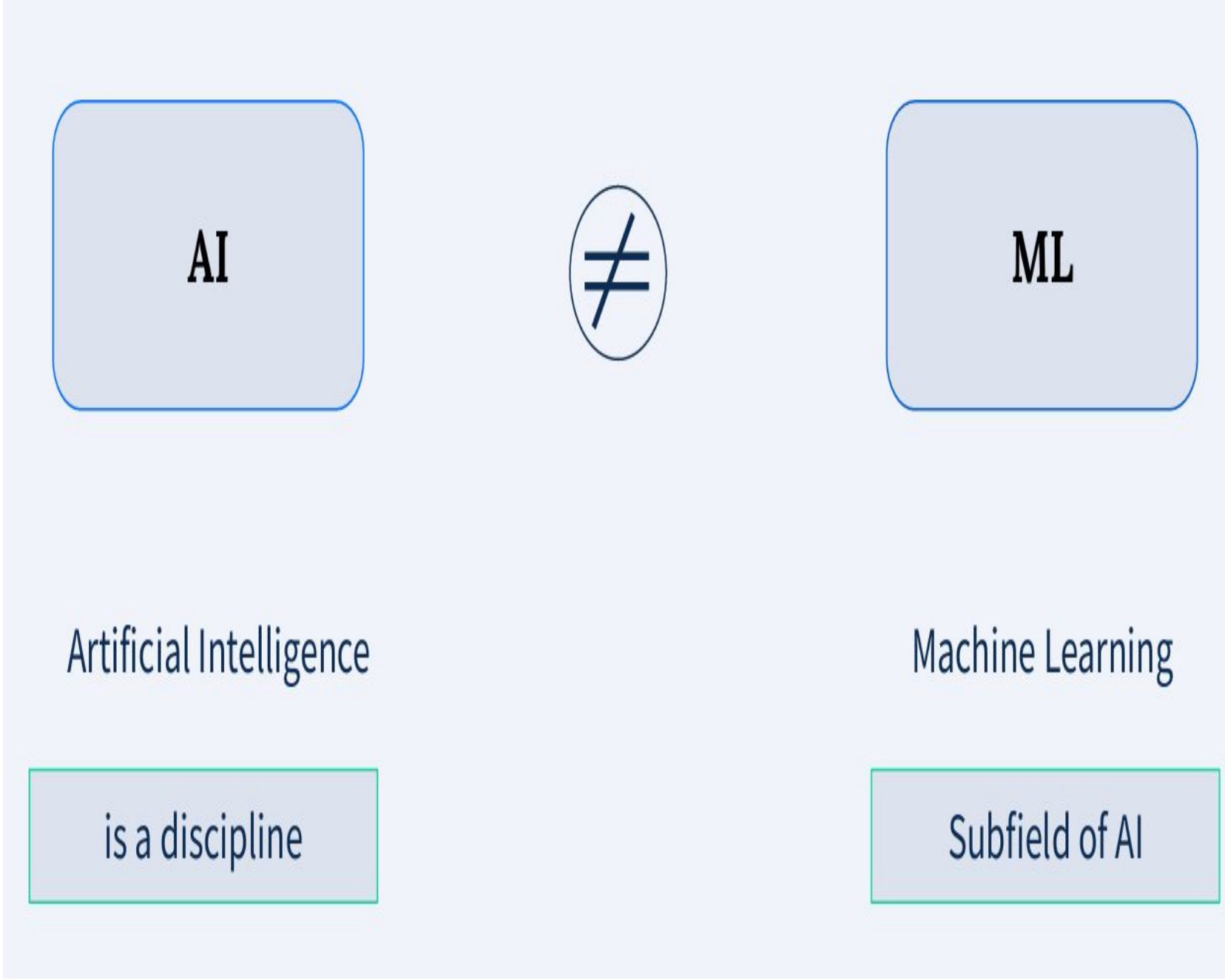
- **These terms are connected, but they do not mean the same thing**

What Does Each Term Mean?

- **1. Artificial Intelligence — AI**
- **AI = The broad field**
- AI means building computer systems that can perform tasks that normally require human intelligence.
- **Examples:**
- Recognizing faces
- Understanding speech
- Playing games
- Making decisions
- Solving problems
- **Simple idea:**
  👉 **AI = Making machines behave intelligently**

2. Machine Learning — ML

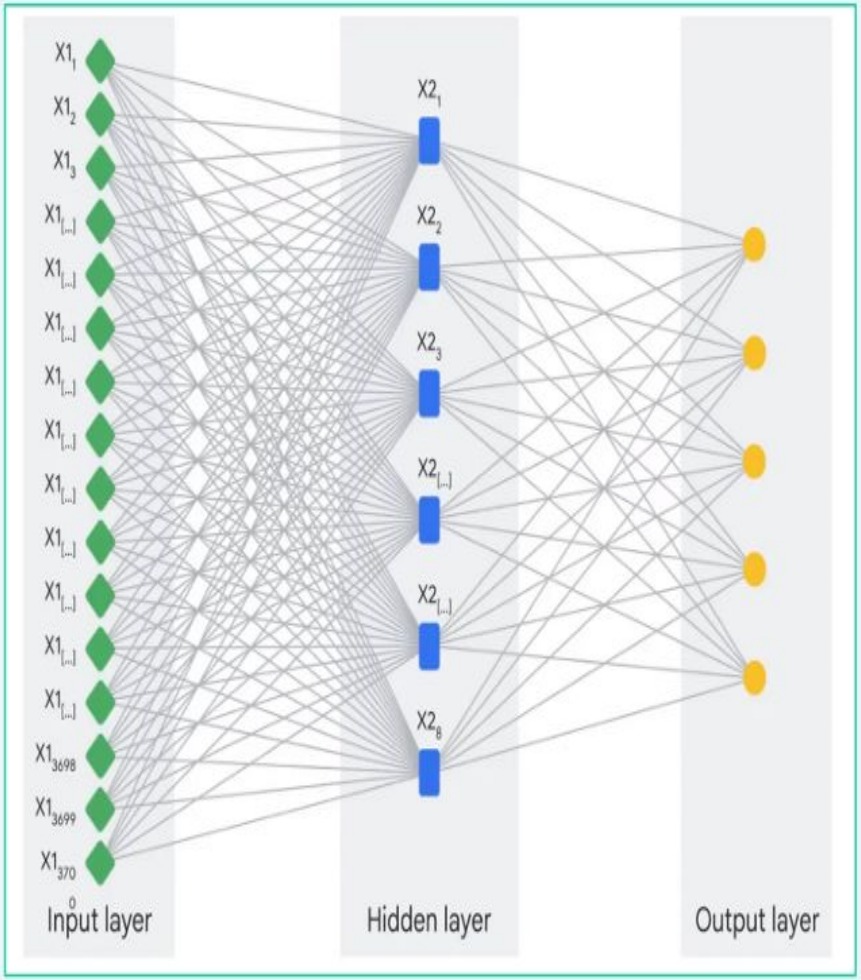
- **ML = A way of building AI systems**
- Instead of programming every rule manually, we give the computer data and allow it to learn patterns.
- **Example:**
- Give the system thousands of emails:
- 📧 Spam
  📧 Not Spam
  📧 Spam
  📧 Not Spam
- The model learns patterns and can then predict whether a new email is spam.
- **Simple idea:**
  👉 **ML = Learning from data**


AI
≠
ML
Artificial Intelligence
Machine Learning
is a discipline
Subfield of AI

Deep Learning

Deep Learning is a Subset of Machine Learning.

It Uses **Artificial Neural Networks** – allowing them to process more complex patterns than Machine Learning.



3. Deep Learning — DL

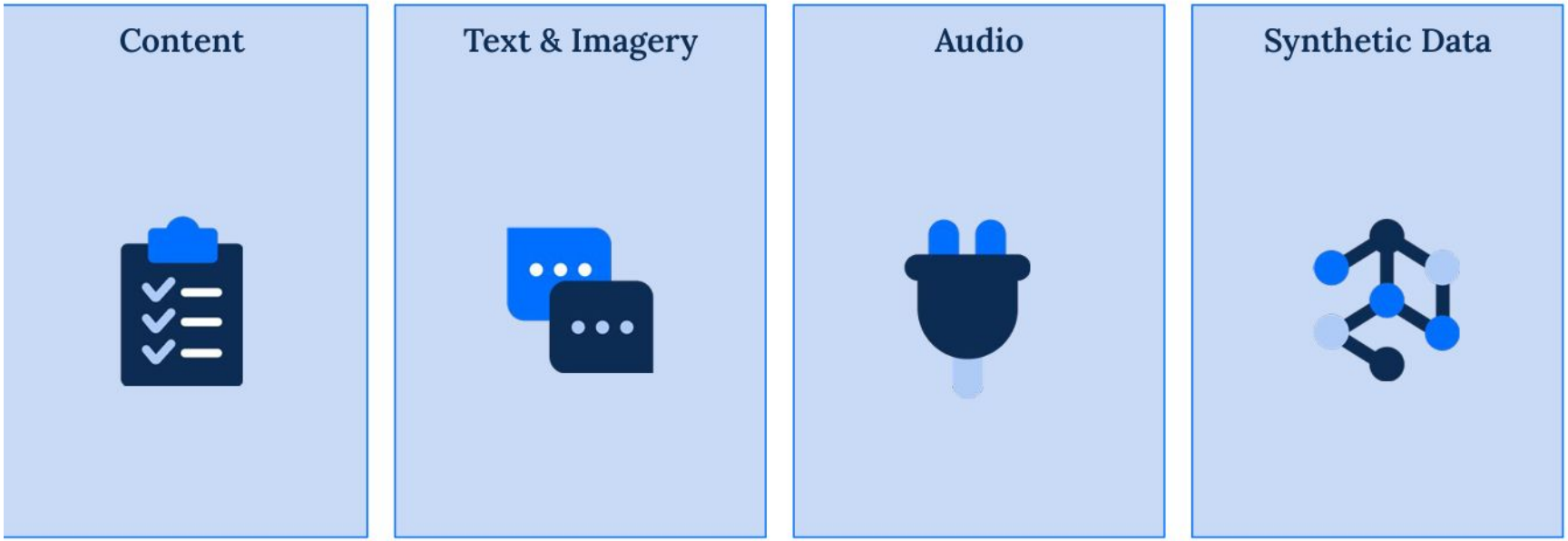
- Example:
- Show a deep-learning model many pictures of cats and dogs.
- Eventually, it can learn patterns that help distinguish:
- 🐱 Cat
  🐶 Dog
- Simple idea:
  👉 Deep Learning = Machine Learning using deep neural networks
-

4. Generative AI — GenAI

- **GenAI = AI that generates new content**
- Generative AI can create:
- ✍️ Text
- 🖼️ Images
- 🎵 Music
- 🎥 Video
- 💻 Code
- 🗣️ Speech
- **Example:**
- You give an AI this prompt:
- “Write a story about a robot visiting Mars.”
- The AI generates a **new story**.
- **Simple idea:**
  👉 **Generative AI = Creating new content**

Generative AI

A type of Artificial Intelligence Technology that can produce various types of





What is Generative AI?

- GenAI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.
- The process of learning from existing content is called training and results in the creation of a statistical model.
- When given a prompt, GenAI uses this statistical model to predict what an expected response might be–and this generates new content.

Main Idea

- AI Broad field of intelligent machines
- ML Learning patterns from data
- Deep Learning ML using deep neural networks
- Generative AI Generating new content

Traditional AI vs Generative AI

Traditional AI

Usually focuses on:

Classification
Prediction
Detection
Recommendation
Decision-making
Example:

“Is this email spam or not?”

Output:

Spam / Not Spam

Generative AI

Focuses on creating new content.

Example:

“Write an email explaining why my meeting needs to be postponed.”

Output:

A newly generated email

Activity

Task	Prediction or Generation?
Is this email spam?	?
Is this image a cat?	?
Write a 200-word story	?
Create an image of a robot	?
Predict tomorrow's temperature	?

Task	Prediction or Generation?
Is this email spam?	Prediction
Is this image a cat?	Prediction
Write a 200-word story	Generation
Create an image of a robot	Generation
Predict tomorrow's temperature	Prediction

What Does “Generative” Mean?

- **Generate = Create**
- Generative AI learns patterns from large amounts of data.
- It can then use those learned patterns to generate new outputs

- **Examples**
- **Input → Generated Output**
- Prompt → Text
- Prompt → Image
- Prompt → Code
- Prompt → Music
- Prompt → Video

What Can Generative AI Generate?

- **Generative AI can work with many types of data**
- Type Examples
- Text Articles, emails, stories
- Images Photos, illustrations, designs
- Audio Speech, music
- Video Scenes, animations
- Code Python, JavaScript, SQL
- Data Synthetic datasets
- 3D Objects and environments

Generative AI in Everyday Life

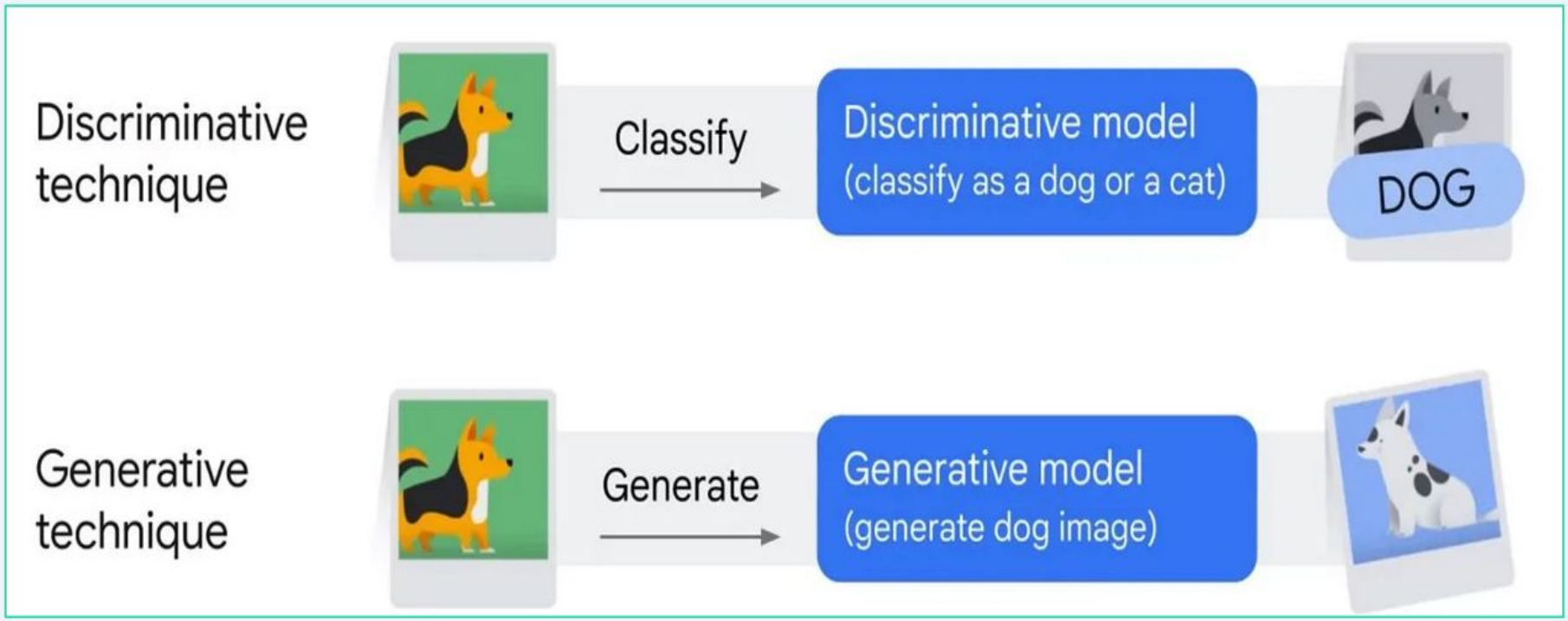
- **Where are we already using it?**
- Chatbots
- Search assistants
- Coding assistants
- Content creation
- Translation
- Education
- Marketing
- Customer support
- Healthcare research
- Software development
- Design
- Business analytics

Generative vs. Discriminative Models

- **Discriminative Models**
- **Goal: Predict / Classify**
- Distinguish between classes
- Learn relationships between input and output
- Example: Spam / Not Spam
- Example: Cat / Dog

Generative vs. Discriminative Models

1. **Discriminative** : Used to Classify or Predict
2. **Generative** : Generate new data that is similar to the data it was trained on. Eg – Next Word Prediction



- **Generative Models**
- **Goal: Generate**
- Learn patterns in data
- Generate new samples
- Example: Generate text
- Example: Generate images

Remember

- **Discriminative → “Which class?”**
- **Generative → “What could a new example look like?”**

- I show a model 100,000 images and ask it to identify whether a new image is a cat or dog. What is the task?

- I show a model 100,000 images and ask it to identify whether a new image is a cat or dog. What is the task?

- Classification / discriminative.

- I give the model the instruction ‘Generate a picture of a cat sitting beside a laptop.’ What is the task?

I give the model the instruction ‘Generate a picture of a cat sitting beside a laptop.’ What is the task?

Generation / generative.

Same Data, Different Goals

- Imagine a dataset containing:
- 🐱 **Cats +** 🐶 **Dogs**
- **Discriminative**
- **Image → Model → Cat / Dog**
- Goal:
- **Classify**
- **Generative**
- **Prompt / Noise → Model → New Image**
- Goal:
- **Generate**
- **Key Idea**
- **The difference is the objective.**

- I will give you a task. You tell me whether it is discriminative or generative

- “Identify whether an X-ray shows pneumonia.”
- “Generate an X-ray-like image for research.”
- “Determine whether a review is positive or negative.”
- “Write a new customer review.”
- “Classify a photograph as indoor or outdoor.”
- “Generate a photograph of a modern classroom.”

- “Identify whether an X-ray shows pneumonia.” **Discriminative / prediction**
- “Generate an X-ray-like image for research.” **Generative**
- “Determine whether a review is positive or negative.” **Discriminative**
- “Write a new customer review.” **Generative**
- “Classify a photograph as indoor or outdoor.” **Discriminative**
- “Generate a photograph of a modern classroom.” **Generative**

What Can Generative AI Generate?

Modality	Examples
Text	Stories, reports, answers
Images	Illustrations, designs
Audio	Speech, music
Video	Clips, animation
Code	Python, Java, SQL

Generative AI is not limited to text.

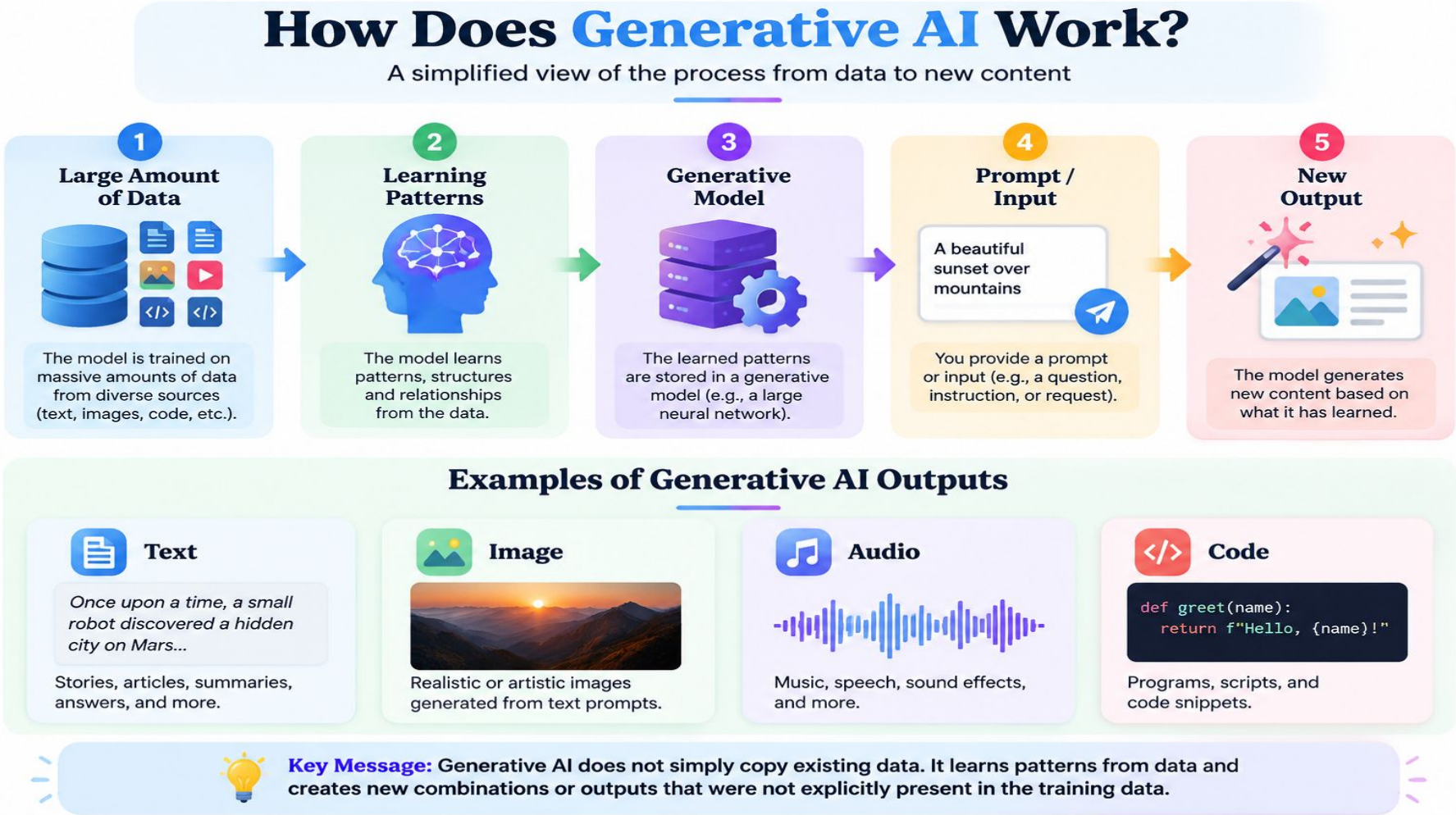
- Is an AI image generator Generative AI?”
- **Yes.**
- “Is a code-generation system Generative AI?”
- **Yes.**
- “Is Generative AI only ChatGPT?”

Generative AI Applications

- **Where Is Generative AI Used?**
- 🎓 **Education**
- 💻 **Software Development**
- 🏥 **Healthcare**
- 🎨 **Design & Creativity**
- 📢 **Marketing**
- 💬 **Customer Support**
- 🔬 **Research**
- ✍️ **Content Creation**
- 📊 **Business**
- 🧪 **Scientific Discovery**

- Where have you already seen Generative AI?

How Does Generative AI Actually Work?



- Imagine you train an AI model with thousands of pictures of cars. What information might the model learn?

- Generative AI is just copying information from the internet.
- Do you agree or disagree?

Training vs Inference

- **Training**
- The model learns from data.
- Example:
- Millions of text examples
  ↓
  Model learns language patterns
- **Inference**
- The trained model generates output.
- Example:
- Prompt:
  "Explain climate change"
- ↓
- Generated answer

- A company says:
- "We improved our AI system because users asked more questions."

- Is this necessarily true?

Generative AI Models Around Us

Examples of Generative AI Systems

Different models. Different strengths. A rapidly evolving ecosystem.

Same idea.... Different capabilities!

System	Main Capability	Example	Typical Use Cases
ChatGPT	Text generation	*"Explain quantum computing in simple words."*	Answering questions Content creation Summarization
Gemini	Multimodal generation	*"Describe this image and suggest a caption."* 	Text, image, video & audio Multimodal understanding Creative and analytical tasks
Claude	Text reasoning	*"Analyze the pros and cons of remote work for students."*	Complex reasoning Long-form analysis Safe and helpful responses
DALL-E	Image generation	*"A futuristic city with flying cars at sunset."* 	Text-to-image generation Concept art Visual content creation
Midjourney	Creative images	*"A serene mountain landscape in watercolor style."* 	Artistic and stylized images Creative design Visual exploration
GitHub Copilot	Code generation	*"Write a Python function to calculate Fibonacci numbers."*	Code completion Code generation Programming assistance

```
def fibonacci(n):
  if n <= 1:
    return n
```

Key Takeaway: Generative AI is not a single model. Different systems are designed and optimized for different types of content, data and use cases.

- A. Generate Python code
- B. Generate image from text
- C. Answer questions using language
- Options:
- Image generation model
- Large Language Model
- Code assistant

- Why do we need different AI models? Why not create one model for everything?

Can Generative AI Make Mistakes?

- Generative AI Is Powerful, But Not Perfect

- Possible problems:
- ❌ Hallucination
- ❌ Bias
- ❌ Incorrect information
- ❌ Privacy risks
- ❌ Copyright issues
- Key principle:
- **AI output must be evaluated.**

Activity: Trust or Verify?

- Would you trust AI output immediately?
- **Case 1**
- AI writes a birthday message.
- Answer:
  Probably yes.
- **Case 2**
- AI gives medical advice.
- Answer:
  Verify.
- **Case 3**
- AI writes code controlling a vehicle.
- Answer:
  Strong verification needed.

Mini Case Study: ChatGPT

From a simple prompt to a detailed explanation

ChatGPT
Ask · Learn · Create · Explore

1 Input

User provides a prompt

> "Explain quantum computing."

This is a natural language input, written in everyday language.

2 Large Language Model

Processes the input using learned patterns

- Trained on massive text data
- Understands context
- Learns language patterns
- Predicts next tokens



ChatGPT
A Large Language Model (LLM)

3 Output

Generated explanation

Quantum Computing

Quantum computing is a new type of computing that uses the principles of quantum mechanics to process information. Unlike classical computers, which use bits (0 or 1), quantum computers use quantum bits, or qubits, which can be in a superposition of 0 and 1 at the same time. This allows quantum computers to perform certain calculations much faster than classical computers, especially for problems in areas such as cryptography, optimization, and scientific simulation.

What the Model Does

At a high level, ChatGPT:

Understands patterns in language

Learns from vast amounts of text data (books, articles, websites, and more).

Predicts likely sequences

Estimates the most probable next words based on the input and context.

Generates responses

Creates coherent, relevant and useful answers, one token at a time.

Key Takeaway: ChatGPT does not simply copy information. It uses learned patterns in language to generate a new and relevant response.

Same technology. Endless possibilities.

- **Netflix Recommendation System** 🎬
- Netflix studies your previous watching history and suggests:
- "You may like this movie.“
- **Question:**
- Is this:
- A) Prediction
  B) Generation

- A student writes:
- "Create a Python program that calculates student grades."
- The AI generates the code.
- **Question:**
- Is this:
- A) Prediction
  B) Generation

- A doctor uploads an X-ray image.
- The AI analyzes the image and says:
- "This scan shows possible signs of pneumonia."
- **Question:**
- Is this:
- A) Prediction
  B) Generation

- **Lecture 1 Final Summary Slide**
- **Today We Learned**
- AI:
  → Intelligent systems
- ML:
  → Learning from data
- Deep Learning:
  → Neural networks
- Generative AI:
  → Creating new content
- Generative models:
  → Learn patterns and generate outputs
- Important question:
- **Is the system predicting something or creating something?**

LEC#2

AI Evolution Timeline

Evolution of Artificial Intelligence

From rules to learning to generation — a journey of continuous progress

1950s — Early AI



Initial attempts to create machines that can simulate human intelligence using logic and rules.

Examples:

- Logic solving
- Game playing
- Theorem proving
- Rule-based programs

1980s — Expert Systems

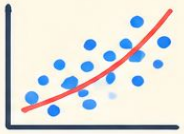


Knowledge from human experts was encoded into rules and logic systems.

Examples:

- Medical diagnosis systems
- Industrial control systems
- Knowledge-based systems

2000s — Machine Learning

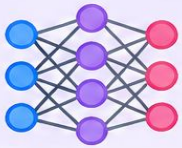


Systems learn patterns from data instead of being explicitly programmed for every rule.

Examples:

- Email spam detection
- Recommendation systems
- Fraud detection
- Prediction models

2010s — Deep Learning



Neural networks with many layers learn complex patterns from large amounts of data.

Examples:

- Image recognition
- Speech recognition
- Natural language processing
- Video analysis

2020s — Generative AI & Foundation Models



Large-scale models that can generate new content and serve as general-purpose foundation models.

Examples:

- ChatGPT (text)
- DALL·E (images)
- Stable Diffusion (images)
- Code assistants
- Multimodal models



Key Takeaway: AI has evolved from manually written rules to data-driven learning, and now to large generative models that can create new content.

Same goal... Smarter methods. Bigger impact.

Why Did AI Keep Changing?

- **Why Did AI Evolve?**
- **1950s–1980s**
- Rule-based AI
- ↓
- Problem:
- Humans had to write every rule
- ↓
- **2000s**
- Machine Learning
- ↓
- Solution:
- Machines learn patterns from data
- ↓
- **2010s**
- Deep Learning
- ↓
- Solution:
- Neural networks learn complex features
- ↓
- **2020s**
- Generative AI
- ↓
- Solution:
- Models create new content

- Why do you think rule-based AI became difficult to maintain?

AROR UNIVERSITY
LEARN · INNOVATE · LEAD

From Rules to Learning

A SHIFT FROM PROGRAMMED KNOWLEDGE TO DATA-DRIVEN INTELLIGENCE

Traditional Expert System

RULE-BASED APPROACH



Human Expert
Provides domain knowledge



Rules
Knowledge encoded as explicit rules (IF–THEN statements)



Computer
Applies rules to new cases



Decision
Outputs a solution or recommendation

Limitations:

- Rules must be written manually
- Difficult to capture all real-world scenarios
- Hard to maintain and scale
- Limited adaptability

Machine Learning

DATA-DRIVEN APPROACH



Data
Examples (labeled or unlabeled data)



Learning Algorithm
Learns patterns from data



Model
Captures learned patterns



Decision
Makes predictions or classifications

Advantages:

- Learns from data automatically
- Can handle complex patterns
- More adaptable to new situations
- Scales with more data
- Improves over time



Key Takeaway: Traditional expert systems rely on human-written rules, while machine learning systems learn patterns from data, making them more flexible, scalable, and powerful.

EVOLVING AI FOR A SMARTER TOMORROW

- Imagine building a medical AI.
- Approach A:
- Write 50,000 medical rules manually.
- Approach B:
- Give the AI millions of medical examples.
- Which approach scales better?

- **1950s: Early AI**
- "Early AI focused on the question:
- Can machines simulate human intelligence?"
- Examples:
- Logic solving
- Game playing
- Rule-based reasoning

- **1980s: Expert Systems**
- "Experts encoded their knowledge into computer rules."
- Example:
- Medical expert system:
- IF patient has fever
  AND cough
  THEN possible flu
- Problem:
- Systems were difficult to maintain because humans had to manually write rules.

- **2000s: Machine Learning**
- "Instead of writing every rule, machines started learning patterns from data."
- Examples:
- Spam detection
- Recommendation systems
- Fraud detection
- Prediction models

- **2010s: Deep Learning**
- "With more data and powerful GPUs, neural networks became extremely effective."
- Applications:
- Image recognition
- Speech recognition
- Natural language processing

- **2020s: Generative AI**
- "Deep learning evolved into large-scale generative models."
- Examples:
- ChatGPT → text
- DALL-E → images
- Stable Diffusion → images
- Code assistants → programming

Activity

- Which stage changed AI from rule-based programming to learning from data.
- Options:
- A. Early AI
  B. Expert Systems
  C. Machine Learning
  D. Generative AI

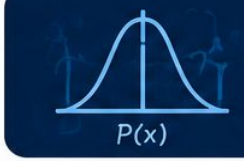
- Why did expert systems become limited compared with machine learning?

- Expert systems:
- Required humans to define rules
- Could not adapt easily
- Machine learning:
- Learns patterns automatically from examples

- A model identifies whether an image contains a human.
- A model creates a realistic image of a human who does not exist.
- What is the difference?

- First: Classification
- Second: Generation

Statistical Generative Models



Statistical Generative Models

First Generation of Generative AI

Modeling Uncertainty Generating Possibilities

Goal

Learn the **probability distribution of data** and generate new samples.

Main Idea

Data
Observed examples

→

Learn Patterns Using Probability
Model the probability distribution

→

Generate New Data
Sample from the learned distribution

Examples

- **Naive Bayes**
  Simple probabilistic model based on Bayes' theorem.
- **Markov Models**
  Model sequential data using probabilities of next states.
- **Hidden Markov Models (HMMs)**
  Handle hidden states in sequential data.

Example: Text Prediction

The model learns word patterns and predicts the next word.

"The weather today is _____."

Possible predictions:

- sunny
- rainy
- cold

Key Takeaway

Statistical generative models introduced the idea that machines can learn data distributions and generate new samples based on probability.

But they have limitations:

- Limited ability to capture complex patterns.
- Require manual feature design.
- Struggle with long-range dependencies.

"From counting patterns to modeling probability — the first step towards generative AI."

Autoencoders

- Input Data
- ↓
- Encoder
- ↓
- Latent Representation
- ↓
- Decoder
- ↓
- Reconstructed Output

- **Main Purpose:**
- Learn important features from data.
- Applications:
- Data compression
- Feature learning
- Noise removal
- Example:
- Image:
- 1000 pixels
- ↓
- Compressed features
- ↓
- Reconstructed image

Variational Autoencoders (VAEs)

- **From Reconstruction to Generation**
- Autoencoder:
- Learns:
- One compressed representation
- VAE:
- Learns:
- A probability distribution

- Process:
- Data
- ↓
- Encoder
- ↓
- Latent Space
- ↓
- Sample
- ↓
- Decoder
- ↓
- New Data
- **Applications:**
- Image generation
- Face generation
- Synthetic data

- A VAE creates a new human face.
- Is this:
- A. Copying an existing face
- B. Generating from learned patterns
- Answer:
- B

Generative Adversarial Networks (GANs)

- Two neural networks compete:
- **Generator**
- Creates fake samples
- **Discriminator**
- Detects:
- Real vs Fake
- Training:
- Generator improves by trying to fool discriminator.
- Applications:
- Realistic images
- Face generation
- Image enhancement



Random Noise

A random vector sampled from a simple distribution (e.g., Gaussian noise).

$z \sim N(0, 1)$

(e.g., 100-dimensional noise vector)

Generator

Neural network that transforms random noise into a fake (synthetic) sample.

$G(z)$

Learn to create realistic data.



Fake Image

Synthetic image generated by the generator.

$G(z)$

A fake sample that looks real.

Discriminator

Neural network that classifies the input as real or fake.

$D(x)$

Outputs a probability that the image is real.

Real / Fake

Discriminator's decision

Real

(P(real) close to 1)

Fake

(P(fake) close to 1)

Lec#3

Foundation Models and the GenAI Ecosystem

- Foundation Models and the GenAI Ecosystem

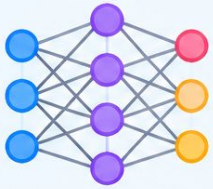
- What are Foundation Models?
- Why are they different from traditional AI models?
- Large Language Models (LLMs)
- Small Language Models (SLMs)
- Open-source vs Proprietary Models
- Training $\longrightarrow$ Adaptation $\longrightarrow$ Inference
- Generative AI Application Lifecycle
- Case study: Building a GenAI application

- **From Large Models to Real-World Applications**
- Evolution:
- Deep Learning
- ↓
- Transformers
- ↓
- Foundation Models
- ↓
- Generative AI Applications

From Large Models to Real-World Applications

The evolution of AI towards real impact

1 Deep Learning



Neural networks with many layers learn complex patterns from data.

Examples:

- Image recognition
- Speech recognition
- Natural language processing
- Computer vision

2 Transformers



Attention mechanism enables better understanding of context and relationships in data.

Examples:

- BERT
- GPT (early versions)
- Language understanding
- Sequence modeling

3 Foundation Models



Large models trained on massive and diverse datasets with general-purpose capabilities.

Examples:

- GPT
- Gemini
- Claude
- Llama

4 Generative AI Applications



Real-world applications that create new content and solve practical problems.

Examples:

- Chatbots
- Image generators
- Code assistants
- Creative tools
- AI-powered solutions

Bigger Models. Broader Capabilities. Real-World Impact.

From Possibilities to Progress

- Is ChatGPT an AI model or an AI application?

Transformer

The architecture behind modern AI

Attention Connects Ideas!

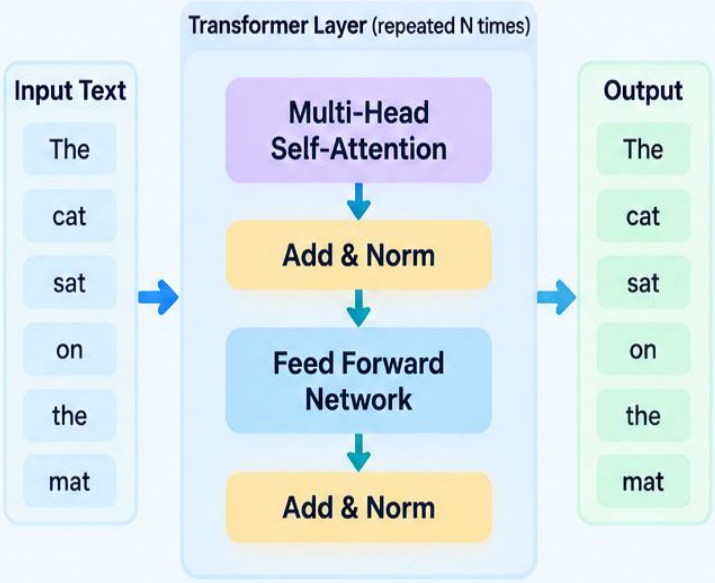
What is a Transformer?

A transformer is a deep learning architecture that uses an **attention mechanism** to understand relationships between all parts of the input data, especially in sequences like text.

Why is it important?

- Understands context better
- Captures long-range relationships
- Enables training of large models
- Powers modern AI systems (e.g., ChatGPT, BERT, etc.)

Basic Transformer Architecture



How Attention Works

Attention allows the model to look at all words in the input and decide **which words are more important** when understanding each word.



When processing "cat", the model can attend to "sat", "mat", and other relevant words, even if they are far away in the sentence.

Key Innovation

Unlike older models (e.g., RNNs), transformers process all tokens **in parallel** using self-attention.

Example: Understanding Context

Sentence:

"The animal crossed the road because it was tired."

What does "it" refer to?

✓ The transformer can use attention to understand that "it" refers to "the animal".

Where are Transformers used?

- ChatGPT (LLMs)
- Search engines
- Translation tools
- Code assistants
- Image generation

Transformers changed AI by allowing models to **understand context**, not just sequence.

Attention makes a bigger picture!

Why Were Transformers Needed?

- "Before Transformers, many language models used Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs)."
- "These models processed words sequentially."
- Example:
- Word 1 → Word 2 → Word 3 → Word 4
- Problems:
- Difficult to capture very long relationships
- Slow training
- Limited scalability
- "Transformers solved these problems by processing multiple words at the same time using attention."

Attention Mechanism

- "The key innovation of Transformers is the **self-attention mechanism**."
- "Self-attention allows the model to calculate how much importance each word should give to other words."
- Example:
- Sentence:
- **"I went to the bank to deposit money."**
- The word **bank** is connected to:
- money
- deposit
- The model understands that this means a financial institution.
- But:
- **"I sat near the bank of the river."**
- Now bank is connected to:
- river
- water
- The meaning changes because the context changes.

Transformer Architecture

A simple step-by-step explanation

Same idea. Massive impact!

1 Input Processing

Divide text into tokens

The input text is divided into smaller units called tokens.

Example

Sentence:

"AI is powerful"

Tokens:

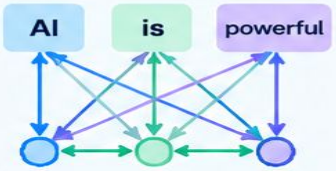
AI | is | powerful

2 Attention and Processing

Understand relationships

The Transformer analyzes relationships between tokens using self-attention.

Self-Attention (simplified view)



The model learns:

- ✓ Which words are related
- ✓ Which information is important
- ✓ How concepts connect

Each token looks at all other tokens to decide what is most relevant.

3 Output Generation

Generate the next token or answer

The model uses this understanding to generate the next token or answer.

Example

Input:

"Explain artificial intelligence"

Transformer

Generated output:

"Artificial intelligence (AI) is the simulation of human intelligence in machines that can learn, reason, solve problems, and make decisions."

Overall Flow



Key Advantages

- ✓ Processes all tokens in parallel
- ✓ Captures long-range relationships
- ✓ Understands context better
- ✓ Scales to very large models
- ✓ Powers modern AI systems (e.g., GPT, BERT, Gemini)

Transformers changed AI by understanding context, not just sequence.

Attention connects words. Transformers create understanding.

- Example:
- Sentence:
- "AI is powerful"
- The model breaks it into:
- AI | is | powerful

Example:

- Sentence:
- "The student opened the book because it was interesting."
- Question:
- What does **"it"** refer to?
- The book.
- "Humans understand this because we use context."
- "Transformers use attention to identify these relationships mathematically."

Example:

- Input:
- "Explain artificial intelligence"
- The Transformer processes the request:
- Input
- ↓
- Transformer
- ↓
- Generated explanation
- Output:
- "Artificial intelligence is the field of creating machines that can perform tasks requiring human intelligence."

- Transformers provided the foundation, but training one Transformer model for a single task is inefficient."
- "The next breakthrough was the creation of **Foundation Models**, where very large Transformer models are trained once and adapted for many applications."

What is a Foundation Model?

- **Foundation Models**
- A foundation model is:
- A large AI model trained on massive and diverse datasets that can be adapted for many tasks.

Foundation Model

A foundation model is a large AI model trained on extensive data, adaptable for downstream tasks like sentiment analysis, image captioning, and object recognition.



Characteristics:

- ✓ Large-scale training
- ✓ General-purpose capability
- ✓ Transfer learning
- ✓ Adaptable to different applications
- Examples:
- GPT
- Gemini
- Claude
- Llama

Traditional AI vs. Foundation Models

From one task to many possibilities

Traditional AI

Typically designed and trained for a single, specific task.



One model
Built for a specific purpose

One task
Solves a single problem



Spam detection
Classifies emails as spam or not spam

Limited to one task.
A new model is needed for a different task.

Foundation Model

A large model trained on diverse data that can be adapted for many tasks.



One model
Large, general-purpose model



Many tasks
Can be adapted for multiple applications



Writing
Create content



Translation
Translate languages



Coding
Write and explain code



Question answering
Answer questions

Flexible and versatile.
One model, many possibilities.

- Why are foundation models called "foundation"?

How Foundation Models Work

- **Foundation Model Pipeline**
- Large Dataset
- ↓
- Pre-training
- ↓
- Foundation Model
- ↓
- Adaptation
- ↓
- Applications
- Example:
- Text data
- ↓
- Large Language Model
- ↓
- Chatbot

- **Example**
- GPT:
- Pre-trained on:
- Books
- Websites
- Articles
- Code
- Adapted for:
- Chat assistant
- Coding assistant
- Research assistant

- Can a foundation model answer questions immediately after pre-training?

Large Language Models (LLMs)

- LLMs are foundation models specialized for language.
- They learn:
- Words
- Grammar
- Context
- Relationships between concepts
- Examples:
- GPT
- Claude
- Gemini
- Llama
- Applications:
- Chatbots
- Translation
- Summarization
- Code generation

- **Example**
- Input:
- "The capital of France is"
- Model predicts:
- "Paris"

- Why does an LLM need billions of parameters?

- To capture complex language patterns.

Small Language Models (SLMs)

- **Small Language Models (SLMs)**
- Smaller models designed for efficiency.
- Advantages:
- ✓ Faster
- ✓ Cheaper
- ✓ Lower hardware requirements
- ✓ Better privacy
- Limitations:
- Less general knowledge
- Lower reasoning ability

- **Example**
- LLM:
- General university assistant
- SLM:
- Only answers questions about university admission rules

- Which model would you choose?
- Scenario:
- A mobile keyboard predicting next words.
- LLM or SLM?

LLM vs. SLM Comparison

Different sizes, different needs, different strengths

Right model for the right task!

Feature	Large Language Models (LLMs) Large-scale, general-purpose models	Small Language Models (SLMs) Smaller, efficient, task-specific models
Size	**Very large** (billions of parameters)	**Smaller** (millions to a few billion parameters)
Cost	**High** Expensive to train and run	**Low** Cost-effective
Speed	**Slower** Higher computational requirements	**Faster** Lower computational requirements
Knowledge	**Broad** General knowledge across many domains	**Limited** Focused, task-specific knowledge
Deployment	**Cloud** Typically deployed in cloud environments	**Edge devices** Can run on local/edge devices (e.g., mobile, IoT)

Examples of LLMs

- **ChatGPT** (GPT-4)
- **Gemini** (Google)
- **Claude** (Anthropic)
- **Llama** (Meta)

Examples of SLMs

- **Phi** (Microsoft)
- **Mistral Small** (Mistral AI)
- **Gemma** (Google)
- **TinyLlama** (Community)

Key Takeaway: Choose the right model based on the task requirements, available resources, and deployment needs.

Activity

- **Choose LLM or SLM**
- Ask students:
- **Case 1:**
- A company wants an AI assistant that writes reports, answers questions, and helps programmers.

- **Case 2:**
- A smartwatch needs a small AI model to monitor health signals.

- **Case 3:**
- A bank wants an AI system working on private customer data inside its own servers.

- Is an SLM always worse than an LLM?

- Is an SLM always worse than an LLM?

- SLMs can be better when:
- The task is specific
- Low cost is required
- Fast response is important
- Data privacy matters

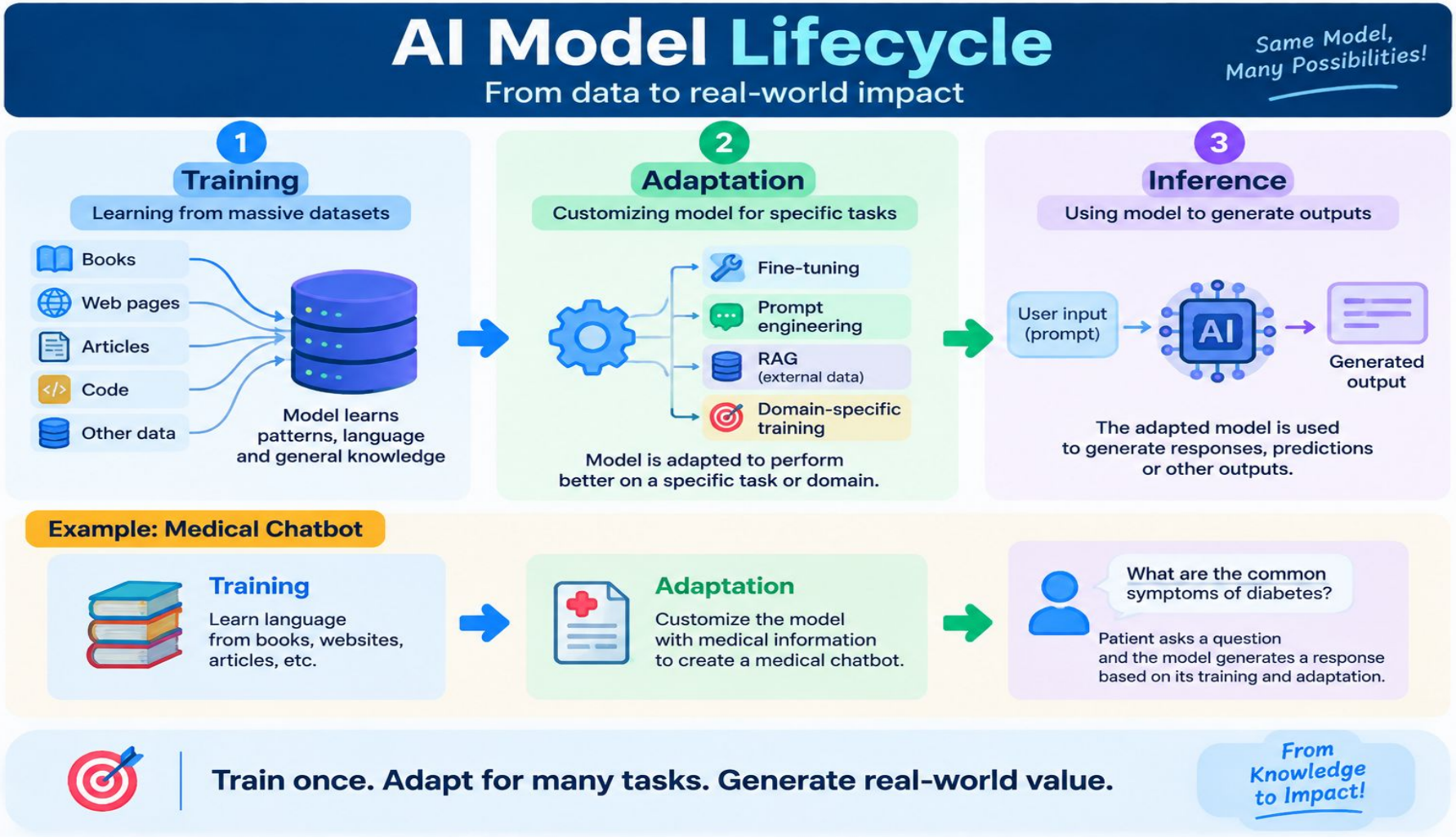
Open vs Proprietary Models

Open vs Proprietary Models

- **Open Models**
- Model weights available publicly.
- Examples:
- Llama
- Mistral
- Advantages:
- ✓ Customization
- ✓ Research access
- ✓ Local deployment

- **Proprietary Models**
- Controlled by companies.
- Examples:
- GPT
- Claude
- Gemini
- Advantages:
- ✓ Easy access
- ✓ Managed infrastructure
- ✓ High performance

Training, Adaptation, and Inference



Activity

- Classify:
- Model learns from books.
- ?
- Model learns medical terminology.
- ?
- User asks medical question.
- ?

Activity

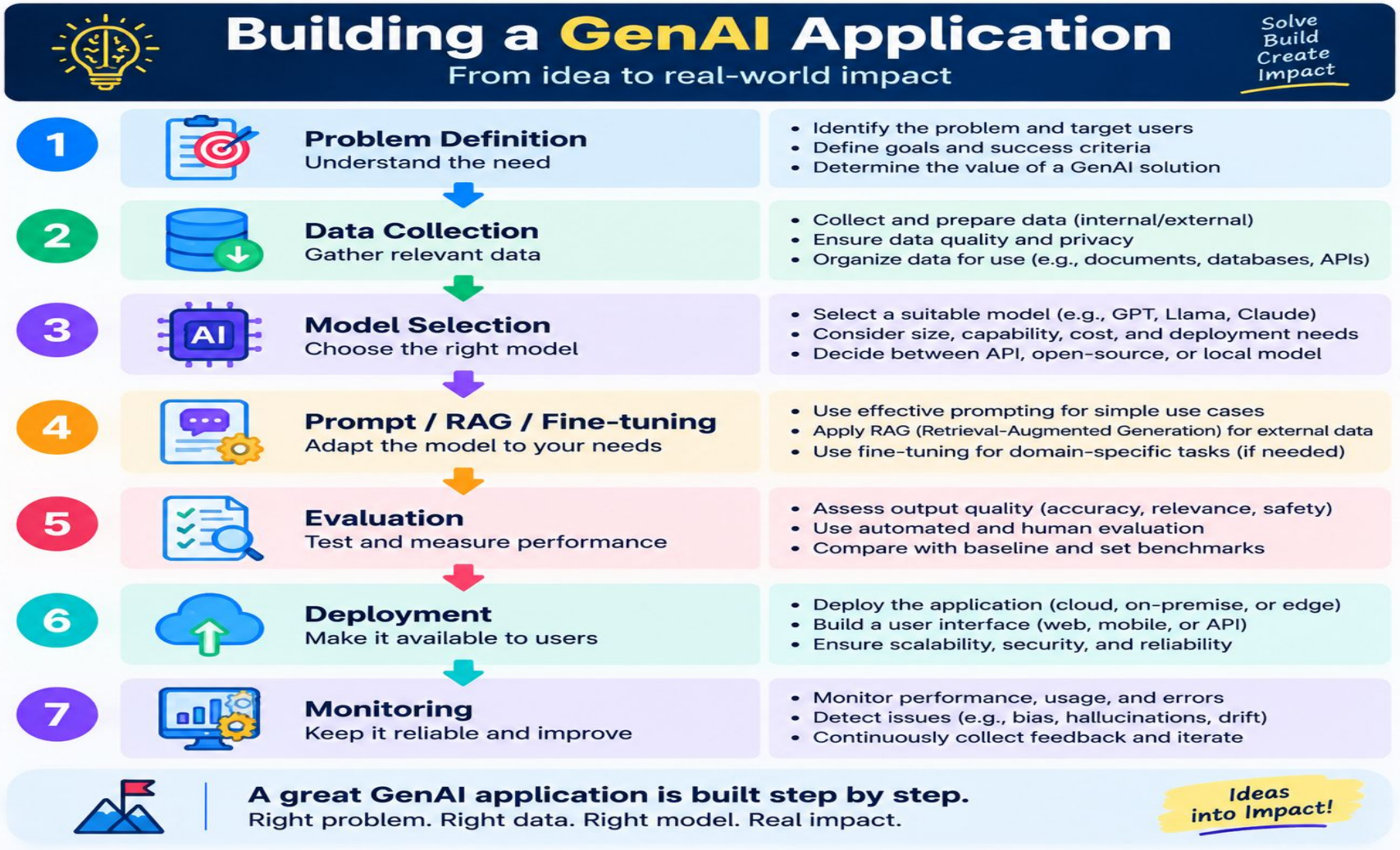
- Classify:
- Model learns from books.
- Training
- Model learns medical terminology.
- Adaptation
- User asks medical question.
- Inference

How Do We Adapt Foundation Models?

- Methods:
- **Fine-tuning**
- Train model further on specific data
- **Prompt Engineering**
- Design better instructions
- **RAG**
- Connect model with external knowledge

- Example:
- General LLM
- ↓
- Add company documents using RAG
- ↓
- Company assistant

Generative AI Application Lifecycle



- **Example**
- University AI Assistant:
- Problem:
- Answer student questions
- Data:
- University regulations
- Model:
- LLM
- Adaptation:
- RAG
- Deployment:
- Web chatbot

Case Study: ChatGPT Ecosystem

- **ChatGPT as a GenAI Ecosystem**
- Foundation Model:
- GPT
- ↓
- Training
- Large-scale text data
- ↓
- Adaptation
- Human feedback + alignment
- ↓
- Application
- ChatGPT assistant

Designing a GenAI Application

- Problem
- Users
- Data source
- Model type
- Adaptation method

- **Example: Designing a GenAI Application**
- **Problem:**
- **University students need quick answers about course information, assignments, and academic policies.**

- **Users:**
- **Students**
- Who will use the system?
- Undergraduate students
- New students
- Online learners

- **Data Source:**
- **University Knowledge Base**
- Data can include:
- Course outlines
- Lecture slides
- University regulations
- Assignment guidelines
- FAQs

- **Model Type:**
- **Large Language Model (LLM)**
- Example:
- GPT
- Llama
- Gemini
- Why?
- Because students need:
- Question answering
- Explanation
- Natural conversation

- **Adaptation Method:**
- **Retrieval-Augmented Generation (RAG)**
- How it works:
- Student Question
- ↓
- Retrieve relevant university documents
- ↓
- LLM generates answer
- ↓
- Student receives response

Activity

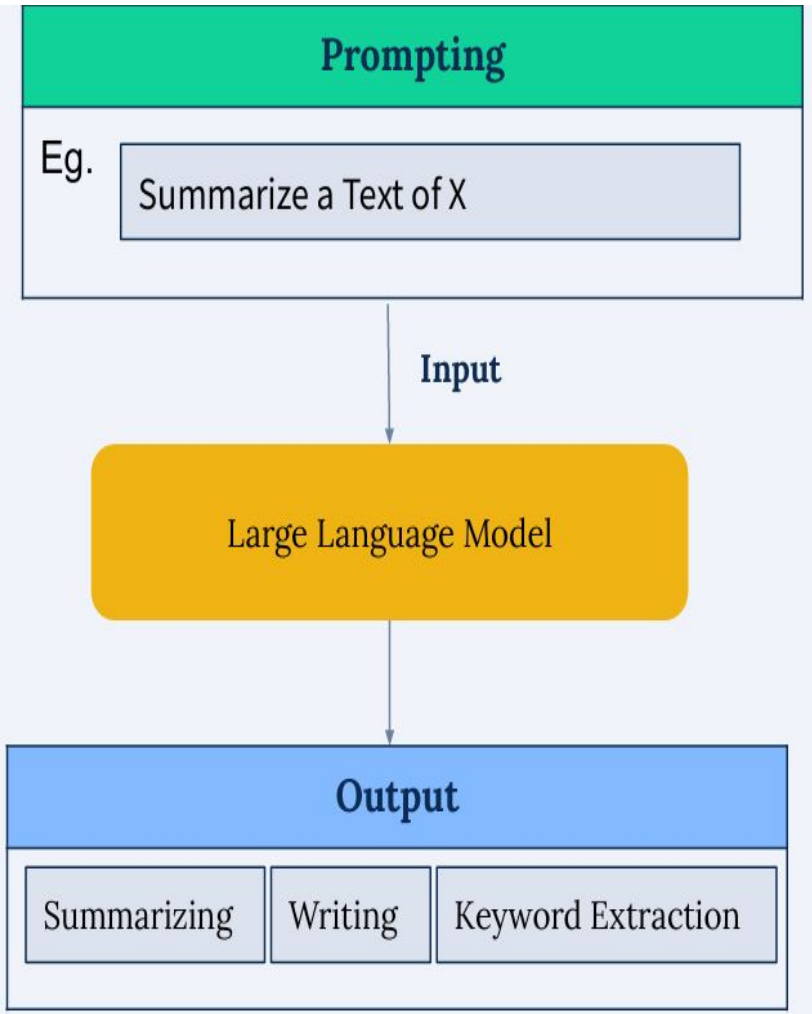
- **Design Your Own GenAI Application**
- Students create their own idea:
- **1. Problem**
- What problem will your AI solve?
- **2. Users**
- Who will use it?
- **3. Data Source**
- What information does it need?
- **4. Model Type**
- LLM or SLM?
- **5. Adaptation Method**
- Prompting, RAG, or Fine-tuning?

Component	Your Idea
Problem	
Users	
Data Source	
Model Type	
Adaptation Method	
Expected Output	

Prompt Design

Prompt Design

The **quality of Input** data determines the The **quality of Output** data



Final Activity

- **Task**
- Imagine you are an AI engineer. Design a GenAI system for a real-world problem.
- Complete these 6 questions:
- What problem will your AI solve?
- Who are the users?
- **What type of content will your AI generate?**
- Choose one:
- ☐ Text
  ☐ Image
  ☐ Audio
  ☐ Video
  ☐ Code

- **Is your system Prediction or Generation?**
- Example:
- Student asks:
- "What are the CLOs of this course?"
- AI creates:
- A natural language explanation.
- Answer:
- Generation
- **What data does the AI need?**
- Examples:
- Lecture notes
- Books
- Websites
- Images
- Documents

- **What risks should we consider?**
- Examples:
- Wrong information
- Bias
- Privacy
- Human verification

Example Answer

- **AI Course Assistant**
- **Problem:**
- Students need quick answers about course materials.
- **Users:**
- Students
- **Content Generated:**
- Text explanations
- **Task:**
- Generation
- **Data:**
- Lecture slides + course documents
- **Risks:**
- AI may provide incorrect answers, so students should verify important information.